
IERM: Compact Interactive Endomorphic Reasoning Models for Program Induction*

Imed Magroune
CEA, Université Paris-Saclay
imed.magroune@cea.fr

Abstract

Many reasoning tasks demand deducing latent laws from few demonstrations to solve unseen problems. It remains unclear whether compact neural architectures, through appropriate inductive biases, can rival the reasoning abilities that large models achieve through scale.

We introduce **Interactive Endomorphic Reasoning Models (IERM)**, a compact neural architecture that explicitly separates two computational roles: the induction of latent programs from demonstrations and their execution during iterative reasoning.

IERM organizes computation into three interacting representation spaces: a support-induced *program space* encoding transformation laws inferred from examples, a compact *reasoning space* serving as a working memory, and a *solution space* responsible for constructing candidate outputs. These components interact through recursive cross-attention updates, enabling progressive refinement of solutions while maintaining compact internal representations.

Despite using very small models, IERM achieves around 12% solved tasks on ARC-AGI-1 and over 63% on Sudoku Extreme using only 2–4M parameters. These results suggest that reasoning ability does not necessarily require massive model scale, but can emerge from appropriate architectural inductive biases and recursive computation.

1 Introduction

Neural networks have achieved remarkable progress in perception, language, and multimodal tasks. However, structured reasoning problems remain challenging. Tasks that require discovering and applying transformation laws—such as abstract puzzles, combinatorial constraints, or dynamical systems—often expose the limits of purely feedforward pattern-matching approaches. Benchmarks such as the Abstraction and Reasoning Corpus (ARC) [Chollet(2019)], Sudoku, and maze planning are specifically designed to evaluate generalization through rule discovery from a small number of examples.

Recent work has explored architectures that incorporate iterative computation or recursive reasoning loops to better capture such structure. In particular, Hierarchical Reasoning Models (HRM) [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Abbasi-Yadkori] and their simplified variant Tiny Recursive Models (TRM) [Jolicoeur-Martineau(2025)] demonstrate that compact neural networks can perform multi-step reasoning by recursively updating latent states representing predictions and internal reasoning context. In these archi-

*Code available at <https://github.com/Imag2020/IERM>

IERM architecture

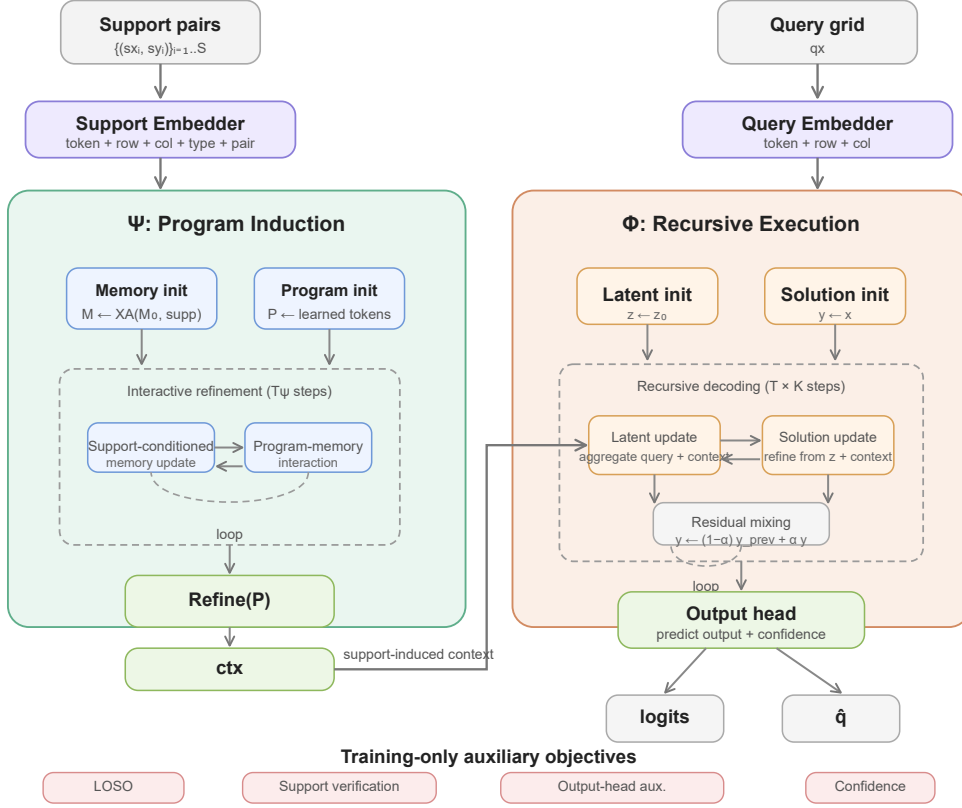


Figure 1: **IERM architecture**. The program induction module Ψ extracts a support-induced latent program from demonstration pairs. The recursive execution module Φ then iteratively refines the query solution using latent reasoning tokens and the induced context. The model predicts both output logits and a confidence score.

tectures, the transformer representation is split into two coupled latent states—a prediction state y and a reasoning state z —that are iteratively refined through a shared stack of layers.

Inspired by this paradigm, we hypothesize that separating program induction from neural execution provides a structural inductive bias for learning algorithmic transformations from examples. We propose **Interactive Endomorphic Reasoning Models (IERM)**, a neural architecture that factorizes computation into three interacting latent spaces: a *program space* encoding transformation laws inferred from demonstrations, a *reasoning space* serving as compact grid-independent working memory, and a *solution space* representing the spatial structure of the candidate output. Rather than reasoning within a single shared representation, IERM separates law induction, abstract reasoning, and solution construction into specialized components coupled through iterative cross-attention. At each iteration the reasoning state aggregates information from the query, the current hypothesis, and the inferred program; the resulting representation then conditions the refinement of the spatial solution state. Through repeated composition of this transformation, the model progressively constructs solutions to tasks requiring multiple intermediate reasoning steps.

By constraining reasoning to a small set of latent tokens independent of grid size, IERM encourages abstraction of transformation rules rather than direct spatial reasoning.

We evaluate IERM across several families of structured reasoning problems defined on two-dimensional grids: combinatorial reasoning (Extreme Sudoku), abstract rule induction (ARC), spatial planning (Hard Maze navigation), and controlled synthetic environments

(cellular automata, PDE heat diffusion). Despite using extremely compact models (2–4M parameters), IERM achieves competitive results, including around 12% accuracy on ARC-AGI-1 and over 63% on Sudoku Extreme. These results suggest that recursive architectures with explicit structural separation can address diverse reasoning problems within a unified framework while remaining highly parameter-efficient.

Contributions. The main contributions of this work are:

- **Interactive Endomorphic Reasoning Models (IERMs)**, a neural architecture that separates computation into program induction, latent reasoning, and spatial solution construction.
- An **endomorphc reasoning formulation** in which a compact recursive architecture (2–4M parameters) iteratively applies a learned transformation on a joint reasoning–solution state.
- An empirical evaluation across diverse reasoning domains including combinatorial puzzles, spatial planning problems, abstract rule induction tasks, and dynamical grid systems.

2 Related Work

2.1 Iterative and Recurrent Computation

Several extensions of the transformer architecture have explored forms of recurrence and iterative computation. The Universal Transformer [Dehghani et al.(2019)Dehghani, Gouws, Vinyals, Uszkoreit, and Kaiser] introduces recurrent application of the same transformation across depth, allowing iterative refinement of representations. Deep Equilibrium Models [Bai et al.(2019)Bai, Kolter, and Koltun] interpret deep networks as fixed-point iterations and provide a theoretical framework for implicit recurrent computation. In the context of large language models, prompting strategies such as Chain-of-Thought [Wei et al.(2022)Wei, Wang, Schuurmans, et al.], Tree of Thoughts [Yao et al.(2023a)Yao, Yu, Zhao, et al.], and Re-Act [Yao et al.(2023b)Yao, Zhao, Yu, Du, Shafraan, Narasimhan, and Cao] encourage explicit intermediate reasoning steps, though these approaches express reasoning through generated text rather than within neural latent states. In contrast, IERM performs iterative reasoning directly within structured latent representations operating over spatial inputs.

2.2 Program Induction and Algorithm Learning

A long line of work has explored neural architectures capable of learning algorithmic procedures. Early examples include Neural Turing Machines [Graves et al.(2014)Graves, Wayne, and Danihelka] and Memory Networks [Weston et al.(2015)Weston, Chopra, and Bordes], which augment neural networks with external memory structures. Neural Programmer–Interpreters [Reed and de Freitas(2016)] and differentiable program synthesis approaches [Gaunt et al.(2017)Gaunt, Brockschmidt, Kushman, and Tarlow] attempt to learn compositional program-like structures directly from data. More recent hybrid symbolic–neural systems such as DreamCoder [Ellis et al.(2021)Ellis, Wong, Nye, et al.] combine program search with neural guidance and progressively learn libraries of reusable program fragments. Neural program synthesis has also been explored for ARC-style tasks, where models attempt to infer explicit transformation programs consistent with example pairs [Ainooson et al.(2023)Ainooson, Sanyal, Michelmore, and Aha]. IERM shares the goal of learning reusable transformation laws from examples, but represents them as latent operators that interact with spatial reasoning states through iterative neural execution rather than generating explicit symbolic programs.

Recent work has also explored learning operators that map between structured states, particularly in the context of physical simulation. Neural operator methods such as DeepONet and Fourier Neural Operators [Lu et al.(2021)Lu, Jin, Pang, Zhang, and Karniadakis, Li et al.(2021)Li, Kovachki, Azizzadenesheli, Liu, Bhattacharya, Stuart, and Anandkumar]

learn mappings between functional representations of dynamical systems. While these approaches focus on continuous physical processes, IERM can be interpreted as learning a discrete reasoning operator acting on latent states conditioned by demonstrations.

2.3 Recursive Reasoning Architectures and ARC

The Abstraction and Reasoning Corpus (ARC) [Chollet(2019)] evaluates the ability to infer transformation laws from a small number of demonstrations and apply them to unseen queries, emphasizing systematic generalization over large-scale memorization.

The work most closely related to ours is the family of compact recursive reasoning architectures proposed for such tasks. Hierarchical Reasoning Models (HRM) [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Abbasi-Yadkori] introduce hierarchical recurrent modules that iteratively refine intermediate representations. Tiny Recursive Models (TRM) [Jolicoeur-Martineau(2025)] demonstrate that very small networks can solve structured reasoning problems through repeated application of shared transformations. These results highlight the importance of recursion as an inductive bias for algorithmic reasoning.

IERM adopts the recursive refinement principle introduced by these models but differs in its architectural factorization: instead of maintaining reasoning within a single latent state, IERM explicitly separates program induction, reasoning dynamics, and solution construction into three interacting spaces. This separation allows reasoning to occur in a compact latent state while maintaining spatial structure only where required, leading to small models that remain competitive with larger recursive architectures.

IERM can also be interpreted as a neural analogue of classical program induction frameworks, where transformation rules are first inferred from examples and subsequently executed by a learned interpreter.

3 Interactive Endomorphic Reasoning Models

3.1 Problem Setting and Architecture

We consider reasoning tasks defined over spatial grids, where the goal is to infer a transformation from a small set of examples and apply it to a new query instance. Let $\mathcal{S} = \{(s_i^x, s_i^y)\}_{i=1}^{N_s}$ denote a set of support pairs, each consisting of an input grid and its corresponding output. Given a query input q^x , the objective is to predict the output grid q^y .

IERM decomposes computation into three interacting latent spaces:

- A **program space** $P \in \mathbb{R}^{n_p \times D_\psi}$, encoding transformation laws inferred from the support set through a program induction module $P = \Psi_\theta(\mathcal{S})$. Each support pair is embedded using a spatial encoder $E(\cdot)$ and aggregated through cross-attention to produce the program tokens. The program representation is not supervised directly; it is shaped implicitly through output consistency, acting as a latent hypothesis explaining the observed transformation.
- A **reasoning space** $z_t \in \mathbb{R}^{n_z \times D_\phi}$, a fixed set of latent tokens serving as grid-independent working memory. At each reasoning step, the reasoning state aggregates information from the current candidate solution, the query input, and the program representation. Its independence from the spatial resolution encourages abstract rather than spatially explicit reasoning.
- A **solution space** $y_t \in \mathbb{R}^{L \times D_\phi}$ with $L = H \times W$, where each token corresponds to a spatial position in the output grid. The solution state is initialized from the embedded query input $y_0 = E(q^x)$, providing an inductive bias toward copy-preserving transformations commonly observed in ARC-style tasks.

We also denote the embedded query input by $x = E(q^x)$. Reasoning proceeds through iterative interactions between these spaces, allowing the model to progressively refine candidate solutions.

Algorithm 1 IERM Inference

Require: Support pairs $\mathcal{S} = \{(s_i^x, s_i^y)\}_{i=1..S}$, query grid q^x

Ensure: Predicted output grid $\hat{q}$

- 1: Encode supports: $h_{\mathcal{S}} \leftarrow E(s_i^x, s_i^y)$
- 2: Encode query: $x \leftarrow E(q^x)$

Program induction (Ψ)

- 3: Initialize memory M_0
- 4: Initialize program tokens P_0
- 5: **for** $t = 1..T_\psi$ **do**
- 6: $M_t \leftarrow \text{XA}(M_{t-1}, h_{\mathcal{S}})$
- 7: $P_t \leftarrow \text{XA}(P_{t-1}, M_t)$
- 8: **end for**
- 9: $ctx \leftarrow \text{Refine}(P_{T_\psi})$

Recursive execution (Φ)

- 10: Initialize latent tokens z_0
 - 11: Initialize solution tokens $y_0 \leftarrow x$
 - 12: **for** $t = 1..T$ **do**
 - 13: $z_t \leftarrow f_z(z_{t-1}, y_{t-1}, ctx)$
 - 14: $y'_t \leftarrow f_y(y_{t-1}, z_t, ctx)$
 - 15: $y_t \leftarrow (1 - \alpha)y_{t-1} + \alpha y'_t$
 - 16: **end for**
 - 17: $\hat{q} \leftarrow \text{OutputHead}(y_T)$
 - 18: **return** $\hat{q}$
-

3.2 Endomorphic Reasoning Dynamics

The core computation of IERM is an iterative transformation acting jointly on the reasoning and solution states. At each step the model applies:

$$(y_{t+1}, z_{t+1}) = \Phi_\theta(y_t, z_t \mid x, P),$$

where x is the embedded query input and P the latent program. This transformation acts as an endomorphism on the joint state space $(y_t, z_t) \in \mathbb{R}^{L \times D_\phi} \times \mathbb{R}^{n_z \times D_\phi}$, mapping the state back to the same space while progressively refining intermediate hypotheses.

Reasoning consists in the repeated application of this transformation:

$$(y_T, z_T) = f_\theta^{(T)}(y_0, z_0 \mid x, P).$$

3.3 Output Head

After the final reasoning step T , the model produces predictions for each grid cell from the final solution representation $y_T \in \mathbb{R}^{L \times D_\phi}$. We consider two output formulations that share the same reasoning backbone.

The simplest formulation applies a linear classifier independently at each spatial position:

$$p(c_i) = \text{softmax}(W y_T^{(i)}).$$

For tasks where large portions of the input remain unchanged, we use a *copy-rewrite* head that interpolates between copying the input token and generating a new one:

$$p(c_i) = (1 - m_i) p_{\text{copy}}(c_i) + m_i p_{\text{rewrite}}(c_i),$$

where m_i is a learned gating coefficient. Unless stated otherwise, our main results use the copy-rewrite head; the relative impact of the two heads is quantified in the ablation studies (Section 5).

Table 1: Extreme Sudoku performance (2M-parameter model). TTA denotes test-time augmentation with 16 attempts.

Variant	Depth	TTA	Accuracy
IERM-Att (T=3)	18	–	48.1%
IERM-Att (T=4)	24	–	48.5%
IERM-Att (T=4)	24	16	54.8%
IERM-MLP (T=4)	24	16	63.2%

3.4 Training

IERM is trained using a combination of task and auxiliary objectives. Reasoning is performed for a fixed number of iterations T , typically between 2 and 6.

Task loss. The primary objective is the cross-entropy between predicted and ground-truth output tokens: $\mathcal{L}_{\text{task}} = \text{CE}(\hat{q}^y, q^y)$.

Support verification. To ensure consistency between induction and execution, the inferred program is required to reconstruct the support outputs: $\mathcal{L}_{\text{sv}}$.

Leave-one-support-out. To promote generalization, one support example is temporarily removed and predicted from the remaining supports: $\mathcal{L}_{\text{loso}}$.

Total objective.

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{sv}} \mathcal{L}_{\text{sv}} + \lambda_{\text{loso}} \mathcal{L}_{\text{loso}}.$$

The execution module Φ conditions only on the abstract program representation $P \in \mathbb{R}^{n_p \times D_\psi}$, which acts as a fixed-size information bottleneck enforcing strict separation between program induction and execution.

4 Experiments

We evaluate IERM on structured reasoning tasks designed to test rule induction, constraint propagation, and iterative solution construction from few demonstrations. Our evaluation addresses whether compact recursive architectures can learn iterative reasoning from examples alone (RQ1), whether separating program induction from execution provides a useful inductive bias (RQ2), and which architectural components contribute most (RQ3, Section 5).

We consider three families of reasoning problems: combinatorial constraint solving (Extreme Sudoku), abstract rule induction (ARC-AGI), and spatial planning (Hard Maze pathfinding). We additionally study the architecture on controlled synthetic environments with known transformation laws (Conway cellular dynamics, PDE heat diffusion); these results are reported in Appendix A. Unless otherwise specified, experiments use models with approximately **2M parameters** (Sudoku, Maze) or **4M parameters** (ARC).

4.1 Extreme Sudoku

Sudoku is a combinatorial reasoning problem requiring the satisfaction of global constraints across rows, columns, and sub-grids. Solving difficult instances requires propagating constraints iteratively across the grid.

IERM achieves strong performance on the Extreme Sudoku benchmark, with the MLP variant reaching over 63% accuracy under test-time augmentation (Table 1). Increasing recursive depth consistently improves performance, supporting the hypothesis that iterative refinement plays a central role in solving constraint-based puzzles.

4.2 ARC-AGI

The Abstraction and Reasoning Corpus (ARC) evaluates the ability to infer transformation laws from a small number of demonstrations and apply them to unseen tasks.

Table 2: Reasoning performance versus model size. IERM uses $\sim 2\text{M}$ parameters for Sudoku and Maze, $\sim 4\text{M}$ for ARC. All results use test-time augmentation where applicable. N/A indicates results not available.

Method	Params	Sudoku	ARC
Direct prediction	27M	0.0	21.0
HRM	27M	55.0	40.3
TRM-Att	7M	74.7	44.6
TRM-MLP	5–19M	87.4	29.6
IERM-Att (ours)	2–4M	54.8	12.5
IERM-MLP (ours)	2–4M	63.2	5.3

Evaluation is performed on the ARC-AGI-1 public evaluation split containing 400 tasks. A task is considered solved only if all query outputs are predicted exactly.

The attention-based model with copy–rewrite head achieves **10.3%** solved tasks on the evaluation split. With test-time augmentation using D_4 geometric symmetries and majority voting across 16 attempts, performance increases to **12.5%**. For completeness, we also report the MLP variant on ARC in Table 2, which reaches **5.3%** — suggesting that attention-based execution is more effective than MLP-based execution for abstract rule induction, even though the reverse holds for the more regular constraint structure of Sudoku.

Although ARC remains challenging, these results demonstrate that compact recursive architectures can acquire non-trivial rule induction capabilities from very few examples.

4.3 Maze Pathfinding

Maze pathfinding provides a stress test for iterative reasoning: solving large mazes requires long-range propagation of connectivity constraints. We evaluate on synthetic 30×30 mazes (1K train, 1K evaluation) where the model must predict a shortest path on a grid of 900 tokens. IERM achieves high per-token accuracy (95.7%) and path recall (0.76), indicating that it captures meaningful spatial structure. However, strict grid-level solving is not reached, suggesting the model learns a probabilistic path likelihood rather than an exact search procedure. When the predicted heatmap is used as a cost function for a classical BFS solver on 10×10 mazes, the solved rate reaches 49%, indicating that IERM captures useful spatial priors despite struggling with global connectivity constraints.

4.4 Comparison with Reasoning Architectures

Table 2 compares IERM with other reasoning architectures. Despite using models that are **2–7 $\times$ smaller** than prior compact recursive architectures, IERM achieves competitive performance across tasks, supporting the hypothesis that separating program induction from execution provides a useful inductive bias.

5 Ablation Studies

We analyze the contribution of the main architectural components of IERM through controlled experiments.

5.1 Architectural Ablations (Maze)

We use the maze task as a controlled benchmark for studying iterative computation, as solving mazes requires propagating spatial constraints across the grid. Table 3 shows that both recursive reasoning and deep supervision have a strong impact on performance. Removing recursion ($T=1$) leads to a large drop in path F1, confirming that iterative refinement is necessary. Replacing the copy–rewrite head with a standard LM head slightly reduces performance, suggesting that the copy bias provides a useful inductive prior for spatial reasoning.

Model variant	Pixel Acc.	Path F1
IERM full	93.1%	0.73
w/o deep supervision	88.4%	0.61
$T=1$ (no recursion)	82.7%	0.52
LM head (no copy)	91.8%	0.69

Table 3: Maze ablations on 10×10 grids.

Augmentation	Solved	Pixel acc
None (baseline)	10.3%	0.872
Color perms ($\times 3$)	9.5%	0.858
Color perms ($\times 5$)	8.5%	0.858
D4 geometry	11.3%	0.870
D4 + color + voting	12.5%	0.868

Table 4: Test-time augmentation on ARC (4M-parameter attention model with copy–rewrite head).

5.2 Reasoning Depth (Sudoku)

The effect of recursive depth on Sudoku is reported in Table 1. Increasing the number of refinement steps improves accuracy, and test-time augmentation provides further gains, with the MLP variant reaching 63.2% under TTA.

5.3 Output Head Ablation (ARC)

On ARC, the choice of output head has a significant impact. The attention-based model with copy–rewrite head achieves **10.3%** baseline accuracy, compared to **5.0%** with a standard LM head, confirming that the copy–rewrite mechanism provides a strong inductive bias for tasks where most grid cells remain unchanged.

5.4 Test-Time Augmentation (ARC)

We evaluate several TTA strategies on ARC, aggregating predictions via majority voting (Table 4). Color permutations alone provide little benefit, suggesting that the model already learns color-invariant representations during training. In contrast, geometric transformations based on the D4 symmetry group (rotations and reflections) improve the solved rate, reaching **12.5%** on the evaluation split.

6 Discussion

The experiments suggest that effective reasoning in neural networks does not necessarily require massive scaling model size, but can instead benefit from architectural structure that separates distinct computational roles.

IERM introduces an explicit factorization between program induction, latent reasoning, and spatial solution construction. This separation creates a structured information flow in which transformation laws are first inferred from support examples, then executed iteratively through a compact reasoning state before influencing the spatial solution representation.

One consequence of this design is that the reasoning space acts as a grid-independent working memory. Because this space does not scale with the resolution of the spatial input, the model is encouraged to represent transformations in an abstract form rather than memorizing grid-specific patterns. The spatial solution state, in contrast, preserves the geometric structure of the task and allows predictions to remain aligned with the input grid.

This separation introduces an implicit information bottleneck between reasoning and spatial representation. The model must compress transformation laws into a small latent reasoning

state before applying them to the spatial solution. Empirically, this constraint appears to encourage the emergence of algorithm-like reasoning behaviour.

The results also highlight the potential of compact recursive architectures for structured reasoning tasks. While architectures such as HRM [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Abbasi-Yadkori] and TRM [Jolicoeur-Martineau(2025)] demonstrate that small recursive networks can solve certain ARC-style tasks, IERM shows that additional structural separation between induction and execution can further improve parameter efficiency.

For example, IERM achieves performance on Extreme Sudoku comparable to HRM while using substantially fewer parameters. These results suggest that explicitly separating law induction from iterative execution may provide a useful inductive bias for reasoning in compact neural architectures.

Despite these promising results, performance on ARC remains far from human-level reasoning. ARC tasks often require discovering highly abstract transformations from very few examples, highlighting the difficulty of learning systematic reasoning from limited supervision. Future work may explore improved program induction mechanisms, longer reasoning horizons, or hybrid symbolic–neural extensions of the architecture.

An important observation is that non-trivial reasoning behaviors emerge even in extremely small models (2–4M parameters). This suggests that architectural structure may play a more important role than raw scale for certain classes of reasoning tasks.

An alternative interpretation of the IERM reasoning dynamics is through the lens of operator learning. Recent work on neural operators has shown that neural networks can learn mappings between function spaces that approximate underlying dynamical systems.

In IERM, the recursive execution module ϕ_θ can be viewed as a learned operator acting on the joint reasoning–solution state. The latent program representation P conditions this operator, effectively selecting a transformation law inferred from support examples.

From this perspective, reasoning corresponds to repeatedly applying a learned operator that maps intermediate states to refined hypotheses.

7 Conclusion

We introduced Interactive Endomorphic Reasoning Models (IERMs), a neural architecture that separates computation into three interacting latent spaces: program induction, reasoning, and spatial solution construction. Across multiple structured reasoning domains—including Sudoku, ARC-style abstract reasoning, maze planning, and cellular automata—IERM demonstrates that compact recursive architectures with 2–4M parameters can learn non-trivial transformation laws while remaining highly parameter-efficient.

These results suggest that explicitly structuring latent computation into specialized interacting spaces may provide a useful direction for improving neural reasoning systems. Future work may explore scaling to larger models, adaptive reasoning depth, and extensions to additional modalities.

IERM demonstrates that structured reasoning behaviors can emerge in neural networks with only a few million parameters, highlighting the importance of architectural inductive biases for reasoning tasks.

References

- [Ainooson et al.(2023)Ainooson, Sanyal, Micheltmore, and Aha] James Ainooson, Deepayan Sanyal, John P Micheltmore, and David W Aha. An approach for solving tasks on the abstraction and reasoning corpus. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [Bai et al.(2019)Bai, Kolter, and Koltun] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.
- [Chollet(2019)] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [Dehghani et al.(2019)Dehghani, Gouws, Vinyals, Uszkoreit, and Kaiser] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [Ellis et al.(2021)Ellis, Wong, Nye, et al.] Kevin Ellis, Catherine Wong, Maxwell Nye, et al. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Programming Language Design and Implementation*, 2021.
- [Gaunt et al.(2017)Gaunt, Brockschmidt, Kushman, and Tarlow] Alexander L. Gaunt, Marc Brockschmidt, Nate Kushman, and Daniel Tarlow. Differentiable programs with neural libraries. In *International Conference on Machine Learning*, 2017.
- [Graves et al.(2014)Graves, Wayne, and Danihelka] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [Jolicoeur-Martineau(2025)] Alex Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.
- [Li et al.(2021)Li, Kovachki, Azizzadenesheli, Liu, Bhattacharya, Stuart, and Anandkumar] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric pdes. *ICLR*, 2021.
- [Lu et al.(2021)Lu, Jin, Pang, Zhang, and Karniadakis] Lu Lu, Pengzhan Jin, Guofei Pang, Zhiping Zhang, and George Karniadakis. Learning nonlinear operators via deepnet. *Nature Machine Intelligence*, 2021.
- [Reed and de Freitas(2016)] Scott Reed and Nando de Freitas. Neural programmer-interpreters. In *International Conference on Learning Representations*, 2016.
- [Wang et al.(2025)Wang, Li, Sun, Chen, Liu, Wu, Lu, Song, and Abbasi-Yadkori] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. Abbasi-Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- [Wei et al.(2022)Wei, Wang, Schuurmans, et al.] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [Weston et al.(2015)Weston, Chopra, and Bordes] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.
- [Yao et al.(2023a)Yao, Yu, Zhao, et al.] Shunyu Yao, Dian Yu, Jeffrey Zhao, et al. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023a.
- [Yao et al.(2023b)Yao, Zhao, Yu, Du, Shafran, Narasimhan, and Cao] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.

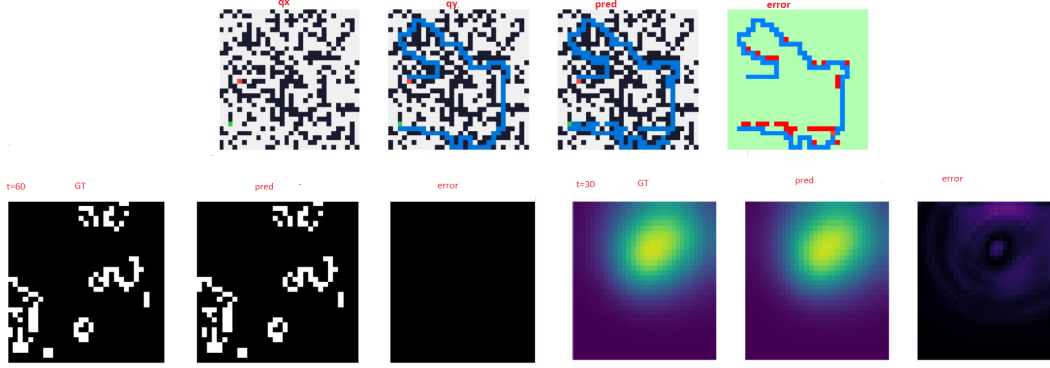


Figure 2: Qualitative inference examples illustrating different aspects of IERM behaviour. **Top:** maze planning. The model typically reconstructs the correct global path, and remaining errors are mostly small local false positives rather than a failure of global reasoning. **Bottom left:** Conway cellular dynamics. The model is trained on one set of transition rules and evaluated on previously unseen rules; in this example it infers the correct rule from support transitions and maintains exact autoregressive rollout over a long horizon. **Bottom right:** PDE heat diffusion. The model captures the large-scale evolution of the field under autoregressive simulation, although small local errors can accumulate over time. The Conway and PDE panels are obtained with the simplified controlled-environment variant used for operator-learning experiments.

A Synthetic Environments

To further analyze the reasoning behavior of the architecture, we evaluate IERM on two synthetic environments where the underlying transformation laws are known. These experiments isolate the program induction and recursive execution mechanisms in a controlled setting.

A.1 Conway Multi-Rule Cellular Dynamics

We evaluate the ability of IERM to infer cellular automaton update rules from demonstrations derived from Conway-style cellular dynamics. Given a small number of support transitions, the model induces a latent program representation that is recursively applied to predict future states.

The model achieves over **80% per-cell prediction accuracy** on evaluation datasets containing rules not seen during training, and remains stable for rollouts of up to 60 time steps.

A.2 PDE Heat Diffusion

We further evaluate IERM on synthetic diffusion dynamics governed by the discrete heat equation:

$$u_{t+1} = u_t + \alpha \Delta u_t.$$

Given a small set of support transitions, the model infers the underlying diffusion operator and recursively applies it to predict future states.

IERM successfully models diffusion dynamics and maintains stable predictions across autoregressive simulations of up to 60 time steps with prediction accuracy exceeding **80%**.

B Implementation Details

B.1 Architecture

Architectural hyperparameters are adapted per task to match its complexity. For ARC, models use a hidden dimension $D_\phi = 256$; for Sudoku and Maze, $D_\phi = 196$. In all configurations the reasoning space uses $n_z = 8$ latent tokens and the program space uses $n_p = 6$ program tokens. The recursive execution module is applied for $T = 3$ –4 reasoning steps depending on the task. Parameter counts are approximately **2M** for Sudoku and Maze, and **4M** for ARC.

The program induction module Ψ_θ processes support examples using shared transformer blocks and produces the latent program representation $P \in \mathbb{R}^{n_p \times D_\psi}$.

The execution module Φ_θ updates the reasoning and solution states through cross-attention interactions between program, reasoning, and spatial tokens.

B.2 Training

Models are trained using the AdamW optimizer with learning rate 2×10^{-4} and weight decay 0.05. Training uses batch size 32 and gradient clipping with norm 0.5. Learning rate warmup is applied for the first 800 steps. Full training recipes, including task-specific schedules, are available in the public code release.

B.3 Datasets

Maze experiments use procedurally generated grids with 1000 training samples and 1000 evaluation samples. Grids are padded to a fixed size of 30×30 tokens.

Sudoku experiments follow the Extreme Sudoku benchmark.

ARC experiments use the official ARC dataset with the standard train/evaluation split.

B.4 Inference and Test-Time Augmentation

For ARC experiments we evaluate several test-time augmentation strategies including geometric transformations from the D_4 symmetry group (rotations and reflections). Predictions are aggregated through majority voting across augmented inputs.